

FoliaScan-Azure

✓ Phase 1 — Build and validate the model locally

The goal of Phase 1 is to build a complete and reliable local machine-learning workflow before introducing cloud infrastructure. This reduces cost and complexity while ensuring that the data, model, training process, and evaluation logic are correct and reproducible.

Here are the 10 most important technical concepts you should understand from **Phase 1**.

1. Data leakage

Data leakage happens when information from validation or test data indirectly enters training.

In FoliaScan, several images may come from the same physical leaf. If one image of a leaf is in training and another image of the same leaf is in testing, the model may partly “recognize” the leaf instead of learning general disease patterns.

That would make the reported accuracy look better than the model really is.

FoliaScan solution: all images with the same leaf_id stay in the same final split.

2. Train, validation, and test splits

Each split has a different purpose:

- Train: used to update model parameters.
- Validation: used to compare epochs and select the best model.
- Test: used once for final evaluation.

The validation set helps answer:

Is the model improving on unseen data?

The test set helps answer:

How well does the final selected model perform on data that played no role in model development?

The test set should not influence checkpoint selection, hyperparameters, or early stopping.

3. Manifest-driven datasets

A manifest is a CSV file describing the dataset.

For FoliaScan, each row contains information such as:

relative_path

class_name

split

leaf_id

source_split

Instead of physically copying images into separate train, validation, and test directories, the program reads the split column.

This makes the dataset:

- Easier to audit
- Easier to reproduce
- Safer to version
- Less wasteful of disk space

The image files remain unchanged while the manifest defines how they are used.

4. PyTorch Dataset and DataLoader

A PyTorch Dataset defines how to load one sample.

For example:

image path

→ open image

→ convert to RGB

→ apply transforms

→ return image tensor and class index

A DataLoader takes the Dataset and creates batches.

For example:

32 images

→ one training batch

The Dataset answers:

How do I load one image?

The DataLoader answers:

How do I efficiently provide batches to the model?

5. Image transforms and augmentation

Transforms prepare images before they enter the model.

Typical deterministic transforms include:

- Resize
- Center crop
- Convert to tensor
- Normalize pixel values

Training augmentation adds small random changes, such as:

- Horizontal flipping
- Small rotations
- Slight crop differences

Augmentation helps the model learn that a disease should still be recognizable when the leaf orientation or framing changes.

Random augmentation is used only for training. Validation and test transforms must remain deterministic so that evaluation is consistent.

6. Class mapping

Neural networks do not directly train with class names such as:

Tomato___Early_blight

They use integer labels:

Tomato___Bacterial_spot → 0

Tomato___Early_blight → 1

...

Tomato___healthy → 9

This mapping must be:

- Deterministic
- Shared by all splits
- Stored in the checkpoint
- Reused during evaluation and inference

A different mapping during evaluation could make correct predictions appear incorrect.

7. Transfer learning and ResNet18

ResNet18 is a convolutional neural network originally designed for ImageNet classification.

The original network predicts 1,000 ImageNet classes. FoliaScan replaces its final classification layer with a layer that predicts 10 tomato conditions.

Transfer learning means starting from a model that has already learned useful visual features such as:

- Edges
- Shapes
- Textures
- Color patterns

Then the model is adapted to the new task.

This is usually faster and more effective than training the entire network from random initialization, especially when the project has limited data or compute.

8. Forward pass, loss, backpropagation, and optimizer

These four parts form the core training cycle.

Forward pass

The images pass through the model, which produces scores for each class:

image batch → model → class scores

Loss

CrossEntropyLoss measures how different the model's predictions are from the correct labels.

Lower loss generally means better predictions.

Backpropagation

Backpropagation calculates how much each trainable model parameter contributed to the error.

It produces gradients.

Optimizer

AdamW uses those gradients to update the model parameters.

The repeated process is:

predict

→ calculate loss

→ calculate gradients

→ update parameters

This happens once for every training batch.

9. Epochs, checkpoints, and early stopping

An epoch means the model has processed the entire training set once.

After each epoch, FoliaScan evaluates the model on the validation set.

Two checkpoints are stored:

- `best_model.pt`: model from the epoch with the lowest validation loss
- `last_model.pt`: model after the most recently completed epoch

The last model is not necessarily the best model.

In your training:

Best epoch: 7

Completed epoch: 10

Validation loss was best at epoch 7, so that checkpoint should be used for final evaluation.

Early stopping ends training when validation loss stops improving for a configured number of epochs. It saves computation and can reduce overfitting.

10. Evaluation metrics and error analysis

Accuracy is the percentage of all predictions that are correct, but it does not explain which classes are difficult.

Important additional metrics are:

Precision

Of all images predicted as a class, how many were correct?

High precision means few false positives.

Recall

Of all real images belonging to a class, how many did the model find?

High recall means few missed examples.

F1-score

F1 combines precision and recall into one score.

Confusion matrix

A confusion matrix shows which classes are confused with each other.

For example:

True: Early blight

Predicted: Late blight

Error analysis

Looking at misclassified images helps identify whether errors come from:

- Visually similar diseases
- Poor image quality
- Background bias
- Incorrect labels
- Weak model generalization
- Highly confident wrong predictions

Metrics tell you how much the model is wrong. Error analysis helps explain why it is wrong.

The complete Phase 1 logic

- Validate the data
- prevent leakage
- create reproducible splits
- load images through a manifest
- apply transforms
- build batches
- adapt ResNet18
- train with loss and backpropagation
- select the best checkpoint using validation
- evaluate once on the untouched test set
- analyze mistakes

Understanding this complete path is more valuable than memorizing many disconnected machine-learning terms.

Here are the most important commands you have used in Phase 1 so far.

Install the project environment

```
poetry install
```

Creates or updates the Poetry virtual environment using:

```
pyproject.toml
```

```
poetry.lock
```

Run project commands through Poetry so they use the correct environment:

```
poetry run <command>
```

Run automated quality checks

```
poetry run pytest
```

```
poetry run ruff check .
```

```
poetry run mypy src
```

They check different things:

`'pytest'`: program behavior and tests

`'ruff'`: formatting and code-quality problems

`'mypy'`: type consistency

All three should pass before committing.

10. Run FoliaScan Python modules

General pattern:

```
poetry run python -m <python-module>
```

Examples from FoliaScan:

Dataset preparation

```
poetry run python -m foliascan.data.cli plantvillage-split `

--source-manifest data/processed/plantvillage_source_manifest.csv `

--output data/processed/dataset_manifest.csv `

--validation-ratio 0.15 `

--random-seed 42
```

Smoke test

```
poetry run python -m foliascan.training.smoke_test `

--manifest data/processed/dataset_manifest.csv `

--data-dir data/raw/plantvillage_tomato_color `

--config configs/training.example.yaml `

--split train `

--device cuda
```

Model training

```
poetry run python -m foliascan.training.train `

--manifest data/processed/dataset_manifest.csv `

--data-dir data/raw/plantvillage_tomato_color `

--config configs/training.example.yaml `

--epochs 10 `

--device cuda `
```

```
--output-dir artifacts/training/resnet18_baseline_v1
```

Model evaluation

```
poetry run python -m foliascan.evaluation.evaluate `
```

```
--manifest data/processed/dataset_manifest.csv `
```

```
--data-dir data/raw/plantvillage_tomato_color `
```

```
--checkpoint artifacts/training/resnet18_baseline_v1/best_model.pt `
```

```
--output-dir artifacts/evaluation/resnet18_baseline_v1 `
```

```
--device cuda
```

The Git workflow to remember

```
git switch main
```

```
git pull --ff-only origin main
```

```
git switch -c feat/new-feature
```

Make and test changes

```
git status
```

```
git diff
```

```
git add <files>
```

```
git diff --staged
```

```
git commit -m "feat: describe the change"
```

```
git push -u origin feat/new-feature
```

◆ **pipx** installs and runs **Python command-line applications** in isolated virtual environments.

For example:

```
pipx install poetry
```

```
pipx install black
```

pipx install ruff

Each tool gets its **own virtual environment**, so its dependencies do not conflict with your projects or other tools. pipx also puts the tool's command on your system PATH, allowing you to run it normally.

◆ **Poetry** is a tool for managing Python projects. It combines several jobs that are often handled separately:

- installing and updating packages
- creating isolated virtual environments
- recording dependencies in `pyproject.toml`
- locking exact package versions in `poetry.lock`
- building and publishing Python packages

Poetry's lock file helps ensure that developers and deployment systems install the same dependency versions.

• 1.1 Project foundation

What was completed

FoliaScan was initialized as a structured Python project using Poetry and a src-based package layout. Testing, linting, and static type checking were configured with pytest, Ruff, and mypy. The repository also established a feature-branch and pull-request workflow for controlled development.

Key learning

A machine-learning project needs more than model code. Dependency management, tests, documentation, configuration files, and a clear repository structure make the work reproducible and easier to maintain.

Review questions

1. Why is a src-based project layout useful?
2. What is the difference between runtime and development dependencies?
3. What different problems do pytest, Ruff, and mypy detect?
4. Why should credentials and generated artifacts be excluded from Git?
5. Why are changes developed on feature branches instead of directly on main?

- **1.2 Generic dataset preparation**

What was completed

Reusable tools were created to discover folder-based image datasets, identify class directories, validate images, report corrupted files, summarize class distributions, and generate deterministic train, validation, and test manifests.

The generic workflow supports common image formats and records metadata such as image dimensions, color mode, extension counts, and class imbalance.

Key learning

Dataset preparation should be treated as a validated pipeline rather than a one-time manual task. Detecting corrupted files, inconsistent classes, and imbalanced data before training prevents silent errors later.

Review questions

1. Why should images be validated before training?
2. What is the benefit of using a manifest instead of moving images into separate folders?
3. Why should dataset splitting be reproducible?
4. How can class imbalance affect a classifier?
5. What is the difference between raw data and processed metadata?

- 1.3 PlantVillage ingestion & safe splitting

What was completed

The official PlantVillage dataset was loaded from Hugging Face, filtered to the Tomato subset, and exported into ten condition classes.

A source manifest was created containing:

```
relative_path  
class_name  
source_split  
leaf_id
```

A second FoliaScan manifest assigned each image to train, validation, or test.

The splitting logic preserves the official test split and groups images by leaf_id, preventing related images of the same physical leaf from appearing in different splits. Five real cross-split leaf conflicts were detected, and test precedence was applied to keep the final dataset leakage-safe.

Key learning

A random image-level split is not always safe. When several images originate from the same physical object, grouping information must be considered. Otherwise, validation and test results may be unrealistically high.

This phase also demonstrated that real external datasets may behave differently from their documentation and therefore require integration testing.

Review questions

1. What does leaf_id represent?
2. Why can image-level splitting cause data leakage?
3. Why was the official test split preserved?
4. What is the difference between source_split and the final FoliaScan split?
5. Why was test precedence used for leaf groups found in both official splits?

- 1.4 PyTorch data pipeline & smoke test

What was completed

A manifest-driven PyTorch pipeline was implemented:

CSV manifest

- dataset records
- PyTorch Dataset
- image transforms
- DataLoader
- ResNet18
- forward pass

The pipeline includes:

Deterministic class-to-index mapping

- Image loading and RGB conversion
- Training augmentation
- Deterministic validation and test transforms
- ImageNet normalization
- Train, validation, and test DataLoaders
- A ResNet18 classifier adapted to ten classes
- A forward-pass smoke test using real images

The expected tensor flow was verified:

Image batch: [batch_size, 3, 224, 224]
Targets: [batch_size]
Model output: [batch_size, 10]

Key learning

A smoke test verifies that data paths, labels, transforms, batches, and model output dimensions work together before spending time on full training.

Review questions

1. What does [32, 3, 224, 224] represent?
2. Why is one shared class mapping required across all splits?
3. Why is augmentation applied only to training images?
4. Why was the final ResNet18 layer changed?
5. What does a smoke test validate that unit tests alone may not detect?

- 1.5 Local baseline training

Purpose

This subphase converts the working data and model pipeline into a reproducible training workflow. The goal is not yet to produce the best possible model, but to establish a dependable local baseline that can later be evaluated, tracked with MLflow, and moved to Azure Machine Learning.

What was completed

The training workflow includes:

- Reproducible random seeding
- Automatic CPU or CUDA device selection
- One-epoch training and validation functions
- Cross-entropy loss
- AdamW optimization
- Sample-weighted loss and accuracy
- Best and last model checkpoints
- CSV and JSON training history
- Early stopping based on validation loss
- Output-directory protection
- A command-line training interface

The workflow follows this structure:

```
Training DataLoader
  ↓
ResNet18 forward pass
  ↓
CrossEntropyLoss
  ↓
Backpropagation
  ↓
AdamW parameter update
  ↓
Validation after each epoch
  ↓
Checkpoint and history files
```

The test split is deliberately excluded from model training and model selection.

Real integration result

The complete workflow was successfully tested for one epoch using the real PlantVillage Tomato dataset and an NVIDIA RTX 3080 Laptop GPU.

Device: CUDA
Number of classes: 10

```
Train loss:      0.9278
Train accuracy:  68.62%
Validation loss:  0.6660
Validation accuracy: 76.17%
```

These values confirm that the training pipeline works end to end. They should not be presented as final model performance because they come from a one-epoch integration check.

Main output artifacts

```
best_model.pt
last_model.pt
history.csv
history.json
```

best_model.pt stores the checkpoint with the lowest validation loss.

last_model.pt stores the model state after the most recently completed epoch.

The history files record training and validation metrics for later visualization and experiment comparison.

Key learning

Training versus validation

The training set is used to update model parameters. The validation set is used to measure generalization and select the best checkpoint. Validation data must never be used for gradient updates.

Forward pass and backpropagation

During the forward pass, the model produces class scores. The loss function measures the difference between those scores and the correct labels. Backpropagation calculates how each trainable parameter contributed to the error.

Cross-entropy loss

Cross-entropy is appropriate for single-label multiclass classification. It penalizes the model when the probability assigned to the correct class is too low.

AdamW

AdamW updates model parameters using gradients while applying weight decay separately. Weight decay helps reduce overfitting by discouraging unnecessarily large parameter values.

Checkpointing

Checkpointing protects progress and makes model selection reproducible. The best checkpoint may occur before the final epoch, especially when validation performance begins to deteriorate.

Early stopping

Early stopping ends training when validation loss stops improving for a defined number of epochs. This reduces unnecessary computation and limits overfitting.

CUDA

CUDA allows tensor operations and neural-network training to run on a compatible NVIDIA GPU. In FoliaScan, GPU acceleration significantly reduces the time required for ResNet18 training.

Review questions

1. Why are gradients calculated during training but not during validation?
2. What does cross-entropy loss measure in a ten-class classification problem?
3. What role does AdamW play after backpropagation calculates the gradients?
4. Why can `best_model.pt` perform better than `last_model.pt`?
5. Why must the test split remain unused during training and checkpoint selection?

- 1.6 Evaluation & error analysis

Planned work

The best checkpoint will be evaluated once on the untouched test split.

The evaluation phase will include:

- Overall test accuracy
- Per-class precision
- Per-class recall
- Per-class F1-score
- Macro and weighted averages
- Confusion matrix
- Misclassified-image report
- Visual error analysis
- Final evaluation summary

Key learning objective

Accuracy alone does not explain which classes the model handles well or poorly. Per-class metrics and confusion matrices reveal systematic weaknesses, such as confusing visually similar tomato conditions.

Review questions

1. Why should the test set be used only after model selection is complete?
2. What is the difference between precision and recall?
3. When is macro F1 more informative than accuracy?
4. What information does a confusion matrix provide?
5. Why should misclassified images be inspected visually?

✓ Phase 2 — Create Azure infrastructure

- 2.1 Azure readiness and subscription setup

Purpose

The purpose of this subphase is to prepare your computer and Azure account before creating cloud resources.

You need to confirm that:

- Azure CLI is installed
- You can sign in
- The correct subscription is active
- Azure Machine Learning CLI commands are available
- Required Azure services are enabled
- Suitable Azure regions are available

This prevents creating resources in the wrong subscription, wrong region, or unsupported environment.

Important concepts and definitions

Azure subscription

A subscription is the main billing, quota, access, and ownership boundary in Azure.

All Azure resources belong to a subscription.

A subscription controls:

- Who pays for the resources
- Available service quotas
- Available regions and services
- User permissions
- Spending limits

Example:

```
Azure subscription
├── Resource group A
├── Resource group B
└── Resource group C
```

Resource group

A resource group is a logical container for related Azure resources.

For FoliaScan, one resource group may contain:

Resource group: rg-foliascan-dev

- |— Azure ML workspace
- |— Storage account
- |— Key Vault
- |— Container Registry
- |— Compute resources

Deleting a resource group normally removes all resources inside it, so resource groups are also useful for cleanup and cost control.

Resource provider

A resource provider represents an Azure service that can create and manage a particular type of resource.

Examples:

Microsoft.MachineLearningServices	→ Azure ML
Microsoft.Storage	→ Storage accounts
Microsoft.KeyVault	→ Key Vault
Microsoft.ContainerRegistry	→ Container Registry

A provider normally needs to be registered in the subscription before its resources can be created.

Azure CLI

Azure CLI is a command-line tool for managing Azure.

It can be used to:

- Sign in
- Select a subscription
- Create resources
- Inspect configurations

- Submit jobs
- Automate cloud operations
- Troubleshoot failures

The main command is:

`az`

Azure ML CLI extension

Azure CLI provides general Azure commands.

The ml extension adds Azure Machine Learning commands:

`az ml`

It provides commands for:

- Workspaces
- Compute
- Data assets
- Environments
- Jobs
- Models
- Endpoints

The relationship is:

```
Azure CLI
├── Azure ML extension
│   └── az ml commands
```

Azure region

A region is the physical geographical area where Azure resources are hosted.

Examples:

Norway East
West Europe
North Europe

Region choice affects:

- Service availability
- GPU availability
- Cost
- Latency
- Data-location requirements
- Quota availability

The nearest region is not always the best region for an ML project.

Active subscription

Your account may have access to several subscriptions, but Azure CLI uses one active subscription at a time.

Commands such as resource creation are performed in the active subscription.

Selecting the wrong subscription may cause:

- Unexpected billing
- Permission errors
- Missing quota
- Resources created in the wrong environment

Important commands

1. Check Azure CLI installation

```
az version
```

Shows the installed Azure CLI version and installed extensions.

Use it when verifying your local Azure environment.

2. Sign in to Azure

```
az login
```

Authenticates Azure CLI using your Azure account.

This is normally the first command before managing resources.

3. List accessible subscriptions

```
az account list --output table
```

Shows the Azure subscriptions available to your account.

Useful columns usually include:

```
Name  
SubscriptionId  
State  
IsDefault
```

Do not publish full subscription or tenant IDs.

4. Show the active subscription

```
az account show --output table
```

Shows which subscription Azure CLI is currently using.

Always check this before creating paid resources.

5. Change the active subscription

```
az account set --subscription "<subscription-name-or-id>"
```

Example:

```
az account set --subscription "Azure for Students"
```

After changing it, verify:

```
az account show --output table
```

6. Install the Azure ML extension

```
az extension add --name ml
```

Adds Azure Machine Learning commands to Azure CLI.

If already installed:

```
az extension update --name ml
```

7. Inspect the Azure ML extension

```
az extension show --name ml --output table
```

Confirms that the extension is installed and shows its version.

8. Test Azure ML commands

```
az ml --help
```

Confirms that Azure Machine Learning CLI commands are available.

It also shows the main Azure ML command groups.

9. Check a resource provider

```
az provider show `
  --namespace Microsoft.MachineLearningServices `
  --query "{Provider:namespace, State:registrationState}" `
  --output table
```

Shows whether Azure Machine Learning services are registered for the subscription.

10. Register a resource provider


```
az provider register `
  --namespace Microsoft.MachineLearningServices
```

Starts provider registration.

Registration may take some time, so check the status afterward.

11. List workspace-supported regions

```
az provider show `
  --namespace Microsoft.MachineLearningServices `
  --query "resourceTypes[?resourceType=='workspaces'].locations[]" `
  --output tsv
```

Helps identify regions where Azure ML workspaces can be created.

What you should understand technically

By the end of this subphase, you should understand this hierarchy:

```
Azure account
  ↓
Subscription
  ↓
Resource group
  ↓
Azure resources
```

You should also understand the tool relationship:

```
Azure CLI
  ↓
Azure ML CLI extension
  ↓
Workspace, compute, data, job, model, endpoint commands
```

And the creation prerequisites:

```
Correct subscription
+
Registered providers
+
Supported region
+
Required permissions
=
Ready to create Azure ML infrastructure
```

Why this matters for AI/ML jobs

Cloud environment setup

AI engineers are often expected to work with cloud subscriptions, projects, resource groups, and permissions—not only model code.

CLI-based workflows

Companies prefer repeatable commands and scripts over manually clicking through a portal for every task.

Being able to use Azure CLI shows that you can:

- Automate cloud tasks
- Reproduce infrastructure steps
- Debug deployment issues
- Work in CI/CD environments

Cost awareness

Cloud resources can generate costs quickly, especially GPUs.

Checking the subscription, region, resource group, and active resources is part of responsible ML engineering.

Troubleshooting

Many Azure problems are not model problems. They may come from:

- Wrong subscription
- Missing provider registration
- Missing permissions
- Unsupported region

- Insufficient quota
- Missing CLI extension

Recognizing these issues is useful in real production environments.

Job keywords connected to this subphase

These terms may appear in AI/ML job descriptions:

Azure CLI
Azure Machine Learning
Azure subscriptions
Resource groups
Cloud infrastructure
Identity and access management
Infrastructure automation
Resource provisioning
Cloud cost management
Azure regions

What you should be able to explain in an interview

You should be able to say:

Before creating the Azure ML environment, I verified the Azure CLI installation, authenticated my account, selected the correct subscription, installed the Azure ML CLI v2 extension, checked required resource providers, and reviewed supported regions. This helped avoid creating resources in the wrong subscription or using an unsupported service configuration.

You should also be able to explain:

- Why a subscription is different from a resource group
- Why Azure ML requires the ml extension
- Why provider registration matters
- Why region selection affects ML workloads
- Why CLI workflows are more reproducible than only using the portal

Review questions

1. What belongs to a subscription, and what belongs to a resource group?
2. What could happen if the wrong subscription is active?
3. Why does Azure use resource providers?
4. What is the relationship between az and az ml?

5. Why should provider registration be checked before creating a workspace?
6. Why might Norway East not be the best region even when you are located in Norway?
7. Why are CLI commands valuable for production and CI/CD workflows?
8. Which information should not be exposed in public screenshots or documentation?

- 2.2 Resource naming and resource group

Purpose

Create a dedicated Azure resource group for FoliaScan and define a clear naming convention before creating the Azure Machine Learning workspace.

Main decisions

Region: Norway East
Resource group: rg-foliascan-dev-ne
Workspace: mlw-foliascan-dev-ne
Environment: Development

Naming pattern:

<resource-type>-<project>-<environment>-<region>

Example:

rg-foliascan-dev-ne

Important concepts

Resource group

A logical container that organizes related Azure resources such as the ML workspace, storage, Key Vault, and compute.

Naming convention

A consistent naming pattern that makes a resource's type, project, environment, and region easy to identify.

Tags

Key-value metadata used for cost tracking, ownership, filtering, and governance.

Environment

The lifecycle stage of a resource:

dev → development
test → testing
prod → production

Important commands

```
az group create `
  --name rg-foliascan-dev-ne `
  --location norwayeast `
  --tags project=foliascan environment=dev purpose=learning

az group show `
  --name rg-foliascan-dev-ne `
  --output table

az resource list `
  --resource-group rg-foliascan-dev-ne `
  --output table

az configure --defaults `
  group=rg-foliascan-dev-ne `
  location=norwayeast
```

What was learned

- Azure resources should be organized before they are created.
- Clear names make cloud environments easier to manage.
- Tags support cost management and governance.
- A resource group simplifies cleanup because related resources are kept together.
- Deleting a resource group may delete everything inside it.

Job relevance

This subphase demonstrates knowledge of:

Azure Resource Groups
Cloud resource organization
Naming conventions
Environment separation
Resource tagging
Cost management
Azure CLI

Interview explanation

I created a dedicated development resource group for FoliaScan in Norway East. I used a consistent naming convention and tags to support resource organization, cost tracking, environment separation, and future automation.

Review questions

1. Why should development and production resources be separated?

2. How do naming conventions improve cloud management?
3. Why are tags useful for companies?
4. What is the risk of deleting a resource group?

- ## 2.3 Azure Machine Learning workspace

Created a development Azure ML workspace in Norway East using a version-controlled YAML configuration.

Workspace: mlw-foliascan-dev-ne

Resource group: rg-foliascan-dev-ne

Region: Norway East

The workspace provides a central location for managing ML jobs, data assets, environments, models, compute, logs, and deployments.

Supporting services include:

- Storage for data and artifacts
- Key Vault for secrets
- Container Registry for Docker images
- Application Insights for monitoring

Important concepts

- Azure ML workspace
- Declarative YAML configuration
- Supporting Azure resources
- Managed cloud resources
- Provisioning state
- Azure CLI defaults

Important commands

```
az ml workspace create --file infra/azure/workspace.yml
az ml workspace show --name mlw-foliascan-dev-ne
az resource list --resource-group rg-foliascan-dev-ne
az configure --defaults workspace=mlw-foliascan-dev-ne
```

Job relevance

Azure Machine Learning
Cloud resource provisioning
Infrastructure configuration
Azure CLI
YAML
MLOps foundations
Experiment and model management

Review questions

1. Why is an Azure ML workspace not the same as compute?
2. Why is a YAML configuration better than only clicking in the portal?
3. What role does each supporting Azure resource play?
4. Which resources can generate costs even before GPU training begins?

- 2.4 CPU and GPU compute

Purpose

This subphase prepares the cloud machines that Azure Machine Learning will use to run preprocessing, testing, and model-training jobs.

FoliaScan uses separate compute targets because CPU and GPU workloads have different technical requirements and costs.

What was completed

A managed CPU cluster was successfully created in Norway East:

Name:	cpu-fs-dev
VM size:	Standard_DS3_v2
vCPUs:	4
Memory:	14 GB
Tier:	Dedicated
Minimum nodes:	0
Maximum nodes:	1
Idle scale-down:	120 seconds
State:	Succeeded

The cluster scales from zero when a job starts and returns to zero after being idle. This avoids paying for a continuously running VM.

The cluster has public SSH access disabled. Its nodes may still receive public IP addresses, which is acceptable for this development environment but should be reviewed for stricter production security.

GPU investigation

GPU sizes and regional quotas were checked before provisioning.

Norway East did not provide a suitable small training GPU for FoliaScan. West Europe provides:

VM size:	Standard_NC4as_T4_v3
GPU:	1 × NVIDIA T4
GPU RAM:	16 GB
vCPUs:	4
Memory:	28 GB

The subscription has four dedicated cores available for the NCASv3_T4 family, which is enough for one node.

The first GPU creation attempt failed because Azure tried to provision the T4 VM in Norway East, where that VM size is unsupported. This was a **regional configuration problem**, not a lack-of-quota problem.

Current status:

CPU cluster: Complete

GPU cluster: Pending regional configuration

Important concepts

Compute target

A resource where Azure Machine Learning executes a job.

A compute target may be a managed cluster, compute instance, Kubernetes environment, or another supported execution platform.

Compute cluster

A managed group of virtual machines used for submitted jobs.

Azure ML can automatically:

```
start nodes
→ run jobs
→ remove idle nodes
```

This is the preferred option for repeatable training pipelines.

Compute instance

A persistent cloud workstation intended for interactive development, notebooks, and debugging.

It behaves more like a personal development machine and may remain active until manually stopped. FoliaScan does not currently need one.

VM size

A VM size defines the hardware available to one node:

- Number of CPU cores
- System memory
- GPU type and count
- GPU memory
- Local disk capacity

For example, Standard_DS3_v2 is a CPU VM, while Standard_NC4as_T4_v3 includes an NVIDIA T4 GPU.

VM family

Related VM sizes belong to the same hardware family.

Azure quotas are often assigned to the family rather than to one exact VM size.

Standard_NC4as_T4_v3
↓
NCASv3_T4 family

Regional quota

Quota is the maximum amount of a VM family that a subscription may use in one Azure region.

A VM may appear in the available-size list but still be unusable because:

- Its family quota is zero
- Available quota is smaller than the VM's required cores
- Regional physical capacity is temporarily unavailable
- The VM is unsupported in the workspace or compute region

Autoscaling changes the number of active nodes according to job demand.

For FoliaScan:

```
min_instances: 0  
max_instances: 1
```

min_instances: 0 reduces idle costs.

max_instances: 1 prevents accidental multi-node scaling and ensures the workload remains within quota.

Dedicated compute

Dedicated VMs provide more reliable capacity and are not intentionally interrupted by Azure.

They cost more than low-priority nodes but are appropriate for the first complete cloud-training workflow.

Low-priority compute

Low-priority VMs use spare Azure capacity at a lower price.

Azure may interrupt or remove them, so they are better for retryable experiments than for an initial end-to-end validation.

Compute region

The region determines where the VM is physically provisioned.

Region affects:

- Available VM sizes
- Quota
- Price

- Data-transfer latency
- Service capacity

A VM available in West Europe may not be available in Norway East.

Important commands

List existing compute targets:

```
az ml compute list
```

Inspect VM sizes in a region:

```
az ml compute list-sizes --location norwayeast
```

Inspect regional quotas:

```
az ml compute list-usage --location norwayeast
```

Create a cluster from YAML:

```
az ml compute create --file infra/azure/compute/cpu-cluster.yml
```

Inspect one cluster:

```
az ml compute show --name cpu-fs-dev
```

A useful filtering command for GPU sizes is:

```
az ml compute list-sizes `
  --location westeurope `
  --query "[?starts_with(name, 'Standard_N')].{Name:name, Family:family, GPUs:gpus}"
  --output table
```

What I learned

- VM availability and quota are separate checks.
- Quotas are specific to a subscription, region, and VM family.
- A successful workspace does not guarantee that every VM size is supported in its region.
- CPU and GPU workloads should use different compute targets.
- Scale-to-zero is an important cloud cost-control technique.

- Declarative YAML makes compute infrastructure reproducible and reviewable.
- Cloud provisioning failures often come from region, quota, permissions, or capacity—not from ML code.
- Public IP access and public SSH access are different security settings.

Job relevance

This subphase demonstrates practical knowledge of:

Azure ML Compute
 CPU and GPU provisioning
 VM sizes and families
 GPU quota management
 Regional infrastructure planning
 Autoscaling
 Scale-to-zero
 Cloud cost optimization
 Dedicated and low-priority compute
 YAML-based infrastructure
 Compute security settings

These skills are relevant because an ML engineer often needs to select suitable hardware, control cloud costs, diagnose quota failures, and configure repeatable training infrastructure.

Interview explanation

I provisioned a managed Azure ML CPU cluster using a dedicated Standard_DS3_v2 VM with autoscaling from zero to one node. Before provisioning GPU compute, I inspected VM availability and family quotas by region. I selected a cost-conscious NVIDIA T4 option in West Europe and confirmed that the subscription had enough dedicated quota. The initial GPU creation failed because Azure attempted to use the Norway East region, which helped me identify the difference between VM availability, quota, and regional configuration.

Review questions

1. What is the difference between a compute cluster and a compute instance?
2. Why can a VM appear in list-sizes but still fail during creation?
3. What is the difference between a VM size and a VM family?
4. Why are quotas region-specific?
5. Why is min_instances: 0 useful?
6. Why is max_instances: 1 appropriate for FoliaScan?
7. When would low-priority compute be a reasonable choice?

8. Why should CPU preprocessing and GPU training use separate clusters?
9. What caused the first T4 creation attempt to fail?
10. What is the difference between disabling public SSH and disabling node public IPs?

- 2.5 Identity, permissions, and cost controls

Purpose

Review who can access the FoliaScan Azure resources, how Azure ML authenticates to supporting services, and how unnecessary cloud costs are prevented.

What was completed

- User access and inherited Azure roles were inspected.
- The signed-in user has Owner permissions for the learning subscription.
- The Azure ML workspace identity and supporting resources were reviewed.
- No unnecessary role assignments were added.
- Both compute clusters were verified with cost-control settings:

Minimum nodes: 0

Maximum nodes: 1

Idle scale-down: 120 seconds

- A budget alert was considered but not configured because FoliaScan is intended to be a zero-cost learning project.
- No Azure training job will be submitted without confirmed free credits.

Important concepts

Identity

A user, application, or Azure resource that can authenticate.

Authentication

Confirms who an identity is.

Authorization

Determines what that identity is allowed to do.

Azure RBAC

Controls access through:

identity + role + scope

Managed identity

Allows Azure resources to access other services without storing passwords or credentials in source code.

Least privilege

An identity should receive only the permissions required for its responsibilities.

Scope

Defines where a permission applies:

Subscription → Resource group → Individual resource

Budget alert

Sends notifications when spending reaches a threshold, but does not automatically stop resources.

Scale to zero

Removes active compute nodes while no job is running, reducing idle VM costs.

Important commands

```
az ad signed-in-user show
```

```
az role assignment list --assignee <object-id>
```

```
az ml workspace show --query identity
```

```
az ml compute list
```

```
az consumption budget list --resource-group rg-foliascan-dev-ne
```

Key learning

- Authentication and authorization are different responsibilities.
- RBAC permissions depend on both the assigned role and its scope.
- Managed identities are safer than storing credentials in code.
- Owner is practical for a personal learning subscription but is usually too broad for an ML engineer in a company.
- Autoscaling reduces compute costs but does not guarantee that the entire Azure environment is free.
- Budget alerts provide warnings, not hard spending limits.

Job relevance

Azure RBAC
Managed identities
Authentication and authorization
Least-privilege access
Resource scopes
Cloud security
Compute autoscaling
Cloud cost control

Interview explanation

I reviewed the user and workspace identities, inspected Azure RBAC roles and scopes, and avoided adding unnecessary permissions. I also configured both Azure ML compute clusters to scale from zero to one node. Since FoliaScan is a zero-cost learning project, I decided not to submit paid training jobs without confirmed free Azure credits.

Review questions

1. What is the difference between authentication and authorization?
2. What three elements form an Azure RBAC assignment?
3. Why is a managed identity safer than credentials stored in code?
4. Why is resource-group scope safer than subscription-wide access?
5. Why does min_instances: 0 reduce cloud costs?
6. Why does a budget alert not guarantee zero spending?

- 2.6 Local-to-Azure connection test

Purpose

Verify that the local FoliaScan Python environment can securely authenticate, connect to the Azure Machine Learning workspace, and discover existing cloud compute resources without submitting jobs or modifying Azure resources.

What was completed

A read-only connection test was implemented using the Azure ML Python SDK v2.

The test successfully confirmed:

```
Connection: connected
Workspace:  mlw-foliascan-dev-ne
Region:     Norway East
```

Both managed compute clusters were discovered:

```
cpu-fs-dev → AmlCompute → Succeeded
gpu-fs-dev → AmlCompute → Succeeded
```

The command did not upload data, start training, create resources, or activate compute nodes.

Important concepts

Azure ML SDK v2

The Python SDK used to work programmatically with Azure ML workspaces, compute, data, jobs, models, and endpoints.

MLClient

The main Azure ML client connecting Python code to a specific subscription, resource group, and workspace.

DefaultAzureCredential

Authenticates without storing credentials in the source code. During local development, it can reuse the existing Azure CLI login.

Environment variables

Keep configuration such as workspace, resource group, and subscription outside the

application code.

Read-only operation

Retrieves Azure information without creating, updating, or deleting resources.

Control plane

Operations that inspect and manage cloud resources, such as reading workspace details or listing compute targets.

Important commands

```
az login
```

```
poetry run python -c "from azure.ai.ml import MLClient; from azure.identity import D
```

```
poetry run python -m foliascan.cloud.azure_connection
```

Key learning

- Local Python code can access Azure using the same identity authenticated through Azure CLI.
- Credentials and subscription identifiers should not be hardcoded.
- Creating an MLClient object alone does not fully prove connectivity; a real Azure operation must be executed.
- Workspace and compute discovery can be tested without running paid ML workloads.
- Mocked tests can validate cloud-integration logic without requiring internet access or a real Azure account.
- CLI tools and Python SDKs provide two complementary ways to work with Azure.

Job relevance

Azure ML SDK v2
MLClient
DefaultAzureCredential
Cloud authentication
Environment variables
Secure configuration
Azure resource discovery
Control-plane operations
Cloud integration testing

Interview explanation

I implemented a read-only local-to-Azure connection test using the Azure ML SDK v2. The application authenticated through DefaultAzureCredential, connected to the configured workspace using MLClient, and successfully discovered the CPU and GPU compute clusters. Sensitive identifiers were kept outside the source code, and no paid job or resource-changing operation was performed.

Review questions

1. What is the role of MLClient?
2. How does DefaultAzureCredential authenticate during local development?
3. Why should subscription IDs and credentials not be hardcoded?
4. Why is a real workspace read required to confirm connectivity?
5. What is the difference between discovering compute and submitting a job?
6. Why should cloud integration tests use mocks?

✓ Phase 3 — Move and Version the Dataset in Azure

- 3.1 Inspect storage and the default datastore

Purpose

Inspect the Azure ML storage environment before uploading or registering any dataset.

What was completed

The Azure ML workspace contains four managed datastores:

```
workspaceblobstore
workspaceartifactstore
workspacefilestore
workspaceworkingdirectory
```

workspaceblobstore will be used for FoliaScan image and manifest data assets. No separate storage account or datastore is currently needed.

No Azure ML data assets have been registered yet.

The local PlantVillage Tomato dataset was measured:

```
Images: 18,160
Size: 0.516 GB
```

No files were uploaded and no compute node was started during this subphase.

Important concepts

Storage account

The Azure resource that physically stores files and objects.

Datastore

An Azure ML connection to a storage location. Training jobs use the datastore without hardcoding storage credentials.

Data asset

A named and versioned reference to data that Azure ML jobs can reuse.

Default datastore

The storage location Azure ML uses by default when local files are uploaded.

Data versioning

Registering dataset changes as new versions instead of silently replacing existing data.

Key learning

- A datastore is a connection, while a data asset is a versioned reference to data.
- The Azure ML workspace already includes managed storage.
- workspaceblobstore is suitable for image folders and CSV manifests.
- Dataset size should be inspected before cloud upload.
- Azure ML data assets and compute resources are separate.
- Listing storage and data assets is read-only and does not start training.

Important commands

```
az ml datastore list
```

```
az ml datastore show --name workspaceblobstore
```

```
az ml data list
```

```
Get-ChildItem <dataset-path> -Recurse -File
```

Job relevance

Azure Blob Storage
Azure ML datastores
Data assets
Dataset versioning
Cloud data management
Data lineage
Storage cost awareness

Interview explanation

I inspected the Azure ML workspace datastores before uploading data. I selected the existing workspace Blob datastore and measured the local PlantVillage Tomato dataset, which contained 18,160 images and approximately 0.516 GB. No data or compute resources were activated during this inspection.

Review questions

1. What is the difference between a storage account and a datastore?
2. What is the difference between a datastore and a data asset?
3. Why should datasets be versioned?
4. Why inspect dataset size before uploading?
5. Does registering a data asset start a compute cluster?

- 3.2 Define the Azure data-asset structure

Purpose

Define reusable and versioned Azure ML data assets before uploading any data.

What was completed

Three Azure ML data definitions were created:

foliascan-tomato-images	→ uri_folder
foliascan-dataset-manifest	→ uri_file
foliascan-source-manifest	→ uri_file

Each asset has:

- A clear name
- An explicit version
- A local source path
- A description
- Project and source tags

Important concepts

Data asset

A named and versioned reference to data used by Azure ML jobs.

uri_folder

Represents a folder containing many files, such as the 18,160 tomato images.

uri_file

Represents one file, such as a CSV manifest.

Asset version

Identifies a specific state of the dataset. Future changes should normally become version 2, not silently replace version 1.

Separation of data and metadata

The images, final split decisions, and original source metadata are stored as separate assets for clearer lineage and auditing.

Key learning

- YAML makes cloud data definitions reproducible and reviewable.
- Dataset files and dataset metadata serve different purposes.
- Explicit versions improve experiment reproducibility.
- Local paths can later be uploaded and registered through the Azure ML CLI.
- Defining an asset does not itself upload data.

Job relevance

Azure ML data assets
Dataset versioning
uri_folder and uri_file
Data lineage
YAML configuration
Reproducible ML pipelines

Interview explanation

I defined separate versioned Azure ML data assets for the PlantVillage image folder, the leakage-safe FoliaScan manifest, and the original source manifest. This kept training inputs, split decisions, and source metadata independently traceable.

Review questions

1. Why are the images and manifests registered as separate assets?
2. What is the difference between uri_folder and uri_file?
3. Why should data changes create a new version?
4. What information belongs in data-asset tags?

- 3.3 Prepare version-controlled data YAML files

Purpose

Upload the local FoliaScan dataset to Azure Blob Storage and register it as reusable, versioned Azure ML data assets.

What was completed

Local data was checked before upload:

Tomato images:	18,160
Dataset-manifest rows:	18,160
Source-manifest rows:	18,160
Image-folder size:	554.39 MB

Three Version 1 data assets were registered:

foliascan-tomato-images	→ uri_folder	→ Version 1
foliascan-dataset-manifest	→ uri_file	→ Version 1
foliascan-source-manifest	→ uri_file	→ Version 1

Because the YAML definitions used local paths, Azure ML first uploaded the files to the workspace Blob datastore and then registered references to the resulting cloud locations.

Important concepts

Upload

Transfers the physical files from the local computer to Azure Storage.

Registration

Creates a named Azure ML asset that references the uploaded storage location.

Asset version

Identifies a specific dataset state. Future dataset changes should normally be registered as Version 2 rather than replacing Version 1.

AzureML URI

A workspace-aware path referencing data through an Azure ML datastore.

Data lineage

Records which dataset version was used by a training or evaluation job.

Key learning

- Uploading data and registering a data asset are related but different operations.
- `uri_folder` is suitable for unstructured folders such as image datasets, while `uri_file` represents an individual file.
- Data assets allow future jobs to reference friendly names and versions instead of long storage paths.
- Uploading the small manifests first was a useful way to test permissions and YAML configuration before transferring all images.
- No CPU or GPU compute cluster was started during the upload.

Job relevance

Azure Blob Storage
Azure ML data assets
Dataset upload
Dataset versioning
Data lineage
`uri_folder` and `uri_file`
Reproducible ML inputs

Interview explanation

I uploaded the PlantVillage Tomato dataset and registered three independent Version 1 Azure ML data assets: the image folder, the leakage-safe split manifest, and the original source manifest. This separated physical data, model-development decisions, and source metadata while preserving versioning and lineage.

Review questions

1. What is the difference between uploading data and registering a data asset?
2. Why were the images and manifests registered separately?
3. Why should modified data be registered as a new version?
4. What information does an AzureML URI represent?
5. Why were the small manifest assets uploaded before the image folder?

- 3.4 Upload and register versioned data assets

Purpose

Move the local PlantVillage Tomato dataset into Azure Storage and register it as reusable, versioned Azure Machine Learning data assets.

What was completed

The following local data was checked before upload:

Images:	18,160
Image-folder size:	554.39 MB
Dataset-manifest rows:	18,160
Source-manifest rows:	18,160

Three Version 1 data assets were uploaded to workspaceblobstore and registered:

```
foliascan-tomato-images
```

```
→ Version 1
```

```
→ uri_folder
```

```
foliascan-dataset-manifest
```

```
→ Version 1
```

```
→ uri_file
```

```
foliascan-source-manifest
```

```
→ Version 1
```

```
→ uri_file
```

The two small manifests were uploaded first to validate the YAML definitions and Azure permissions before transferring the complete image folder.

Important concepts

Upload

Transfers physical files from the local machine to Azure Blob Storage.

Registration

Creates a named Azure ML reference to the uploaded cloud data.

Data asset

A reusable and versioned input that Azure ML jobs can reference.

Asset version

Identifies one specific state of the data. Updated data should normally become Version 2

instead of replacing Version 1.

uri_folder

Represents a folder containing multiple files, such as the image dataset.

uri_file

Represents one file, such as a CSV manifest.

Data lineage

Records which data asset and version were used by an experiment or training job.

Important commands

```
az ml data create --file <data-definition.yml> az ml data show --name <asset-name> --
```

Key learning

- Uploading files and registering data assets are related but different operations.
- Azure ML jobs can use asset names and versions instead of long storage paths.
- Images, final split decisions, and source metadata should remain separate assets.
- Explicit versions support reproducibility and auditing.
- Uploading data does not start CPU or GPU compute.

Job relevance

Azure Blob Storage
Azure ML data assets
Dataset versioning
Data lineage
uri_folder and uri_file
Cloud data management
Reproducible ML pipelines

Interview explanation

I uploaded the PlantVillage Tomato dataset to the Azure ML workspace datastore and registered three separate Version 1 data assets for the images, final leakage-safe manifest, and original source manifest. This preserved data lineage and allowed future jobs to reference stable, versioned inputs.

Review questions

1. What is the difference between uploading and registering data?
2. Why were the images and manifests registered separately?
3. Why should changed data use a new asset version?
4. What is the difference between uri_folder and uri_file?

- 3.5 Verify data integrity and Azure access

Purpose

Confirm that the registered Azure data assets are accessible and that their metadata matches the verified local dataset.

What was completed

The read-only verification command confirmed:

```
Verification status:  verified
Registered assets:    3
Local images:         18,160
Dataset-manifest rows: 18,160
Source-manifest rows: 18,160
```

For each Azure asset, the tool verified:

- Correct name
- Correct Version 1
- Correct asset type
- Required tags
- Presence of a cloud storage path

The local manifest fingerprints were also calculated:

```
dataset_manifest.csv
SHA-256:
6b67bc229b2f260d8ee4b7eb62e1f64e8ef2e8610ba40f1ff00662d897eddbdfbplantvillage_source_
SHA-256:
4ef1117eb112dd4355958c40447b2536bb265b7bb665d1fc1b1ff3e12685fde6
```

No data was created, modified, downloaded, or deleted during verification. No compute cluster was started.

Important concepts

Data integrity

Confidence that data is complete, consistent, and unchanged.

SHA-256 fingerprint

A hash based on file content. A content change produces a different fingerprint.

Metadata verification

Checks an asset's name, version, type, tags, and cloud-path presence.

Read-only verification

Inspects resources without creating, updating, or deleting them.

Environment variables

Provide Azure configuration without hardcoding subscription or workspace identifiers in source code.

Verification limitation

The tool verified Azure asset metadata and local file integrity. It did not download and compare every cloud image byte-for-byte.

Important command

```
poetry run python -m foliascan.cloud.azure_data_assets `
  --image-root data/raw/plantvillage_tomato_color `
  --dataset-manifest data/processed/dataset_manifest.csv `
  --source-manifest data/processed/plantvillage_source_manifest.csv
```

Local integrity commands:

```
Get-FileHash <file> -Algorithm SHA256Import-Csv <manifest.csv>
```

Key learning

- Successful registration should always be followed by verification.
- Matching counts can reveal missing files or incomplete manifests.
- A SHA-256 hash is stronger evidence of unchanged content than a filename or file size.
- Azure identifiers should be supplied through environment variables.
- Cloud verification logic can be tested with mocked SDK clients.
- Asset metadata verification is not the same as full byte-level cloud verification.

Job relevance

Data quality validation
SHA-256 hashing
Azure ML SDK
Cloud asset verification
Dataset integrity
Secure configuration
Integration testing
Data lineage

Interview explanation

I implemented a read-only verification tool that accessed the Azure ML workspace through the SDK, validated all three registered data assets, confirmed their versions,

types, tags, and cloud paths, and compared them with the local inventory of 18,160 images and manifest rows. I also generated SHA-256 fingerprints for reproducibility.

Review questions

1. Why are file counts alone insufficient for integrity verification?
2. What does a SHA-256 fingerprint detect?
3. What is the difference between metadata verification and byte-level verification?
4. Why should Azure configuration remain outside source code?
5. Why should cloud integration tests use mocks?

✓ **Phase 4 — Train the model in Azure**

- 4.1 Select the Azure ML training environment

Purpose

Choose a reproducible software environment for running FoliaScan training on Azure GPU compute.

Selected environment

`AzureML-acpt-pytorch-2.8-cuda12.6`

This is a Microsoft-curated **Azure Container for PyTorch (ACPT)** environment with a GPU-ready PyTorch and CUDA stack. Microsoft's current PyTorch training example uses this exact curated environment with the `@latest` label.

For initial development:

`AzureML-acpt-pytorch-2.8-cuda12.6@latest`

Before a real reproducible experiment, the resolved environment version should be pinned rather than relying permanently on `@latest`. Azure ML job YAML supports both an exact environment version and an `@latest` reference.

Important concepts

Azure ML environment

Defines the container, operating system, Python runtime, CUDA libraries, and Python packages used by a job.

Curated environment

A Microsoft-maintained, prebuilt environment. Cached curated images generally reduce environment preparation and avoid building a container from scratch.

Custom environment

A project-specific environment created when curated packages are insufficient.

Container image

A packaged Linux runtime that helps the same training code run consistently on cloud compute.

Environment versioning

Preserves the exact software runtime used by an experiment.

Key learning

- Local and Azure environments do not need to be identical.
- The local Poetry environment is currently CPU-only, but the selected Azure environment includes CUDA support.
- Compute defines the hardware; the environment defines the software.
- Curated environments are useful for the first cloud-training workflow.
- An exact environment version is more reproducible than @latest.

Job relevance

Azure ML environments
PyTorch and CUDA
Docker containers
Curated environments
Dependency management
Environment versioning
Reproducible training
MLOps

Interview explanation

I inspected available Azure ML environments and selected Microsoft's curated ACPT PyTorch 2.8 CUDA 12.6 environment for GPU training. I separated the cloud runtime from my local Poetry environment and planned to pin the resolved environment version for reproducible experiments.

Review questions

1. What is the difference between compute and an environment?
2. Why can local PyTorch be CPU-only while Azure training uses a GPU?
3. What is the difference between curated and custom environments?
4. Why is an exact version safer than @latest?
5. When would a custom environment become necessary?

- 4.2 Adapt the training entry point for cloud inputs

Purpose

Prepare the existing local FoliaScan training workflow to run inside an Azure ML command job without creating a separate cloud-specific training implementation.

What was completed

The existing command remains the training entry point:

```
python -m foliascan.training.train
```

It can now accept:

```
Mounted Azure image-folder path
Mounted Azure manifest-file path
Local training configuration file
Azure-managed output-directory path
```

The implementation supports both:

```
Local relative paths
Absolute Linux paths provided by Azure ML
```

The training workflow continues to use only:

```
train split
validation split
```

The test split remains isolated from training.

Azure-compatible inputs and outputs

The CLI supports these path arguments:

```
--data-dir
--manifest
--config
--output-dir
```

All generated training artifacts are written below --output-dir:

```
best_model.pt
last_model.pt
history.csv
history.json
```

This allows Azure ML to collect the output directory as a job output.

Smoke-test controls

Two optional arguments were added:

```
--max-train-batches  
--max-validation-batches
```

Example:

```
poetry run python -m foliascan.training.train `
  --manifest data/processed/dataset_manifest.csv `
  --data-dir data/raw/plantvillage_tomato_color `
  --config configs/training/baseline.yml `
  --output-dir artifacts/smoke-test `
  --max-train-batches 2 `
  --max-validation-batches 1
```

These options limit the number of processed batches without modifying the dataset or manifest.

When omitted, the complete training and validation loaders are processed.

Output-directory protection

The CLI supports an existing empty output directory, which is important because Azure ML may create the output directory before starting the command.

A non-empty output directory is still rejected unless:

```
--overwrite
```

is explicitly supplied.

This helps prevent accidental replacement of previous checkpoints and training histories.

Important concepts

Training entry point

The command Azure ML executes to start model training.

Mounted input

Azure ML makes a registered data asset available as a filesystem path inside the job container.

Managed output

A directory whose contents Azure ML uploads and associates with the completed job.

Portable path handling

Training code accepts both Windows/local paths and absolute Linux/cloud paths without assuming a fixed project location.

Smoke test

A deliberately small execution used to confirm that code, data, environment, and outputs work together before starting full training.

Test-set isolation

The test data is not used during model development, early stopping, or hyperparameter decisions.

Key learning

- Local and cloud training should reuse the same core implementation.
- Azure-specific paths should be passed as arguments instead of hardcoded.
- The cloud platform manages input and output paths, while the Python code only reads or writes those paths.
- Small batch limits make initial cloud validation faster and cheaper.
- Metrics must represent only the samples actually processed.
- Output protection remains important in both local and cloud environments.
- Cloud compatibility can be tested locally without starting Azure compute.

Files updated

```
README.md
docs/training.md
src/foliastan/training/engine.py
src/foliastan/training/train.py
tests/training/test_engine.py
```

tests/training/test_train.py

No separate Azure training module was necessary because the existing training entry point was made portable.

What was intentionally not added

Azure job submission
MLflow tracking
Model registration
Model deployment
Test-set evaluation
Automatic compute startup

These responsibilities belong to later subphases.

Job relevance

Azure ML command jobs
Cloud-mounted datasets
Managed job outputs
Portable training pipelines
PyTorch training engines
Smoke testing
Test-set isolation
Reproducible ML workflows
MLOps

Interview explanation

I adapted the existing FoliaScan training entry point to work with Azure ML mounted inputs and managed outputs instead of creating a separate cloud-only training pipeline. The CLI now accepts arbitrary absolute or relative paths, preserves test-set isolation, writes all artifacts under the provided output directory, and supports limited-batch smoke tests before full cloud training.

Review questions

1. Why is reusing the local training entry point better than creating a separate Azure training script?
2. What is the difference between an Azure ML input and output?
3. Why must absolute Linux paths be supported?
4. Why are smoke-test batch limits useful?
5. Why must the test split remain isolated?

6. Why should a non-empty output directory be protected?

7. What is the difference between limiting batches and changing the dataset?

- 4.3 Define a CPU smoke-test job

Purpose

The goal of this subphase was to create a small and low-cost Azure ML job that verifies whether the complete cloud training pipeline works correctly before using GPU compute.

The smoke test checked that:

- The project code could run inside Azure ML.
- The registered data assets could be mounted.
- The training entry point accepted Azure-mounted paths.
- The selected environment contained the required dependencies.
- The CPU compute target worked.
- Training artifacts were written to an Azure-managed output.

Work Completed

A CPU command-job definition was created:

```
infra/azure/jobs/cpu-smoke-test.yml
```

Main configuration:

```
Compute: cpu-fs-dev
Device: CPU
Epochs: 1
Maximum training batches: 2
Maximum validation batches: 1
```

Registered inputs:

```
foliascan-tomato-images:1
foliascan-dataset-manifest:1
```

Named output:

```
training_output
```

Important Concepts

Smoke Test

A smoke test is a very small execution used to confirm that a system works from beginning to end.

Its purpose is not to measure final model quality. It is designed to detect problems such as:

- Incorrect paths
- Missing dependencies
- Invalid YAML
- Data-mounting issues
- Compute configuration problems
- Output-writing failures

Because only two training batches and one validation batch were processed, the resulting accuracy was not meaningful for model evaluation.

Read-Only Inputs

The data assets were configured with:

```
mode: ro_mount
```

This means Azure ML mounted the registered data into the job container as read-only paths. The training process could read the images and manifest but could not modify the original registered assets.

Managed Output

The output was configured with:

```
mode: rw_mount
```

Azure ML provided a writable directory to the training script and stored the generated files in the workspace storage.

Execution Result

The CPU smoke-test job completed successfully:

```
Job name: heroic_pump_6kvy3nnvpt
Status: Completed
Compute: cpu-fs-dev
```

Generated artifacts:

```
best_model.pt
last_model.pt
history.csv
history.json
```

Smoke-test metrics:

Train loss: 2.1012
Train accuracy: 26.56%
Validation loss: 17.9859
Validation accuracy: 0%
Elapsed training time: 7.96 seconds

These metrics should not be used to judge model quality because only a few batches were processed.

What I Learned

Before running a complete and more expensive GPU training job, it is good engineering practice to validate the entire pipeline using a limited CPU smoke test.

Interview Explanation

I created a low-cost CPU smoke-test job before running full GPU training. It processed only a few batches and verified the Azure ML environment, mounted data assets, training command, compute target, and managed outputs.

- 4.4 Define the GPU training job

Purpose

The goal of this subphase was to define the full Azure ML training job for the ResNet18 tomato-leaf classifier using GPU compute.

Work Completed

A second command-job definition was created:

```
infra/azure/jobs/gpu-training.yml
```

Main training configuration:

```
Compute: gpu-fs-dev
Device: CUDA
Model: ResNet18
Epochs: 10
Batch size: 32
Optimizer: AdamW
Learning rate: 0.001
Weight decay: 0.0001
```

Registered inputs:

```
foliascan-tomato-images:1
foliascan-dataset-manifest:1
```

Named output:

```
training_output
```

CPU Smoke Test vs. GPU Training

Configuration	CPU smoke test	GPU training	Main purpose
			Validate the pipeline Train the real model
Device	CPU	CUDA	GPU
Epochs	1	10	
Training batches	2	All batches	
Validation batches	1	All batches	
Metrics useful for model selection	No	Yes	

Environment Pinning

The curated Azure ML environment was pinned to an exact version:

```
AzureML-acpt-pytorch-2.8-cuda12.6:15
```

Using an exact environment version improves reproducibility. A future job will use the same

software environment instead of automatically changing when a new latest version is released.

Timeout

The GPU job included:

Timeout: 7200 seconds

The timeout provides a safety limit so that a stuck job does not continue running indefinitely.

What the YAML File Controls

The Azure ML YAML definition specifies:

- The submitted project code
- The command to execute
- The registered input assets
- The managed output
- The compute cluster
- The runtime environment
- Environment variables
- Execution limits
- Job name, experiment, description, and tags

What I Learned

A version-controlled YAML job definition makes the cloud-training process repeatable, reviewable, and easier to reproduce than manually configuring every run through the Azure portal.

Interview Explanation

I defined the GPU training workflow as a version-controlled Azure ML command-job YAML file. It used registered data assets, a pinned curated PyTorch environment, managed GPU compute, named outputs, and an execution timeout.

- 4.5 Submit and monitor the training job

Purpose

The goal of this subphase was to submit the command jobs to Azure ML, monitor their execution, inspect the logs, and confirm that both jobs completed successfully.

Execution Order

The jobs were submitted in this order:

1. Submit the CPU smoke test
2. Confirm that the CPU job completed
3. Submit the full GPU training job
4. Stream and inspect the GPU logs
5. Confirm the final GPU job status

This order reduced risk and avoided using GPU compute before the pipeline had been validated.

Job Submission

The jobs were submitted with:

```
az ml job create
```

During submission, Azure ML uploaded the relevant repository code as a job snapshot.

The uploaded code package was approximately:

1.65 MB

Log Monitoring

Live output was monitored with:

```
az ml job stream
```

During the CPU job, the local log stream was interrupted once, but the Azure job continued running.

Important lesson:

Interrupted log streaming does not necessarily mean the Azure job has stopped.

The job status must be checked separately with:

```
az ml job show
```

CPU Job Result

Job name: heroic_pump_6kvy3nnvpt
Display name: foliascan-cpu-smoke-test
Status: Completed
Compute: cpu-fs-dev

The GPU job was submitted only after the CPU status was confirmed as Completed.

GPU Job Result

Job name: cyan_boot_v9vg1214b3
Display name: foliascan-gpu-training
Status: Completed
Compute: gpu-fs-dev
Device: CUDA
Completed epochs: 10
Classes: 10
Early stopped: False

Final GPU Training Metrics

Best epoch: 10
Best validation loss: 0.212930
Best validation accuracy: 0.928964
Final training accuracy: 0.941544

The GPU run completed all ten epochs successfully.

Epoch Results

Epoch	Train accuracy	Validation accuracy	Validation loss
1	68.95%	74.52%	0.8544
2	80.94%	69.57%	1.0062
3	85.50%	83.68%	0.4905
4	88.40%	86.98%	0.4032
5	90.35%	87.03%	0.3641
6	90.85%	89.60%	0.2841
7	91.76%	91.06%	0.2593
8	92.37%	78.51%	0.9935
9	93.40%	90.19%	0.2781
10	94.15%	92.90%	0.2129

Result Interpretation

Overall model performance improved during training:

Training accuracy: 68.95% → 94.15%
Validation accuracy: 74.52% → 92.90%

Validation performance temporarily dropped during epoch 8:

Validation accuracy: 78.51%
Validation loss: 0.9935

However, the model recovered during epochs 9 and 10 and reached its best validation result at epoch 10.

The temporary fluctuation could be related to training randomness, data augmentation, or optimization behaviour.

Runtime Observation

The first epoch took approximately:

780 seconds

Later epochs took approximately:

92 seconds each

The much longer first epoch likely included environment preparation, initial data access, container startup, or initial framework warm-up in addition to model training.

Azure ML Job Lifecycle

The practical workflow observed in this phase was:

```
Submit job
→ Upload code
→ Queue job
→ Provision compute
→ Prepare environment
→ Mount data
→ Run training
→ Write outputs
→ Complete job
→ Scale down compute
```

What I Learned

A job name is an important identifier used to:

- Check status
- Reconnect to logs
- Download outputs

- Retrieve metadata
- Cancel a running job
- Find the run in Azure ML Studio

Interview Explanation

I first validated the workflow with a CPU smoke test and submitted the full GPU job only after the smoke test completed successfully. I monitored both jobs through the Azure CLI, streamed the training logs, and explicitly verified their final statuses.

- 4.6 Track metrics and artifacts

Purpose

The goal of this subphase was to download, inspect, and verify the outputs of the Azure ML jobs.

This included:

- Saving job identifiers
- Downloading named outputs
- Verifying model checkpoints
- Reviewing training metrics
- Finding the best epoch
- Calculating file hashes
- Saving job metadata
- Checking compute scale-down settings

Saving Job Identifiers

The CPU and GPU job names were stored locally:

```
artifacts/azure/job-records/cpu-job-name.txt
artifacts/azure/job-records/gpu-job-name.txt
```

Saving job identifiers makes it easier to retrieve job information later.

Downloading Named Outputs

The outputs were downloaded with:

```
az ml job download
```

The named Azure ML output was:

```
training_output
```

Local structure:

```
artifacts/
├── azure/
│   ├── jobs/
│   │   ├── <job-name>/
│   │   │   ├── named-outputs/
│   │   │   │   └── training_output/
```

Generated Artifacts

Both training jobs produced:

best_model.pt
last_model.pt
history.csv
history.json

best_model.pt

Contains the checkpoint from the epoch with the lowest validation loss.

last_model.pt

Contains the checkpoint from the final completed epoch.

history.csv

Stores one row of metrics per epoch and is useful for:

- Pandas analysis
- Excel inspection
- Plotting
- Reporting

history.json

Stores the same metric history in a machine-readable format suitable for:

- Scripts
- APIs
- Automation
- Further processing

Artifact Verification

All four required GPU artifacts were present.

Approximate checkpoint sizes:

best_model.pt: 128.1 MB
last_model.pt: 128.1 MB

Azure displayed a recommendation to use AzCopy for files larger than 100 MB. This was only

a transfer-performance recommendation; the downloads still completed successfully.

Best Epoch Selection

The metric history was sorted by validation loss.

Best result:

Epoch: 10
Train loss: 0.172537
Train accuracy: 94.15%
Validation loss: 0.212930
Validation accuracy: 92.90%
Epoch training time: 92.49 seconds

Checkpoint Hashes

SHA-256 hashes were calculated for:

best_model.pt
last_model.pt

A cryptographic hash can be used to:

- Verify file integrity
- Detect accidental changes
- Distinguish model versions
- Support reproducibility
- Confirm that a downloaded file matches a recorded artifact

The two checkpoint hashes were different, which is normal because the checkpoint contents were not identical.

Job Metadata

Full job metadata was saved locally as:

job-metadata.json

This metadata includes information such as:

- Job name
- Status
- Compute target
- Environment
- Inputs
- Outputs
- Azure resource paths

- Job configuration

Because job metadata may contain subscription identifiers and internal Azure paths, it should be reviewed before publishing.

Compute Scale-Down

Both compute clusters were configured with:

```
Minimum instances: 0
Maximum instances: 1
Idle time before scale-down: 120 seconds
```

Compute targets:

```
cpu-fs-dev
gpu-fs-dev
```

Setting the minimum instance count to zero allows the cluster to return to zero active nodes after the job finishes and the idle period passes.

This is important for cost control.

Current Metric-Tracking Limitation

Metrics are currently saved in:

```
history.csv
history.json
```

MLflow metric logging has not yet been added.

Therefore, the current phase supports file-based metric tracking but not yet:

- Azure ML metric charts
- MLflow run comparison
- Logged hyperparameters
- Confusion-matrix tracking
- Experiment dashboards
- Model registration through MLflow

These belong to the next project phase.

What I Learned

A trained model alone is not enough for a reliable machine-learning workflow. The following information should also be preserved:

- Dataset version
- Environment version
- Training configuration
- Compute configuration
- Job identifier
- Training metrics
- Best-model checkpoint
- Last checkpoint
- Artifact hashes
- Job metadata

Together, these provide traceability and reproducibility.

Interview Explanation

After training, I downloaded the named Azure ML outputs, verified the required checkpoints and metric-history files, identified the best epoch using validation loss, calculated SHA-256 hashes for artifact integrity, stored the job metadata, and verified that both compute clusters were configured to scale down to zero.

- **Phase 5 — Track experiments with MLflow**
- **Phase 6 — Register the best model**
- **Phase 7 — Deploy an online endpoint**
- **Phase 8 — Add a simple application**
- **Phase 9 — Add GitHub Actions**